

Complete Time Complexity

O(1):-

```
n=100
i = 1
count=0
while i<=n:
    count=count+1
    if i==1:
        break
    i=i+1
print(count)

#O/P:-
if n=10      O/P = 1
if n=100     O/P = 1
if n=1000    O/P = 1
# i.e time complexcity is o(1)
```

O(n)

```
n=100
i = 1
count=0
while i<=n:
    count=count+1
    i=i+1
print(count)

#O/P:-
if n=10      O/P = 10
if n=100     O/P = 100
if n=1000    O/P = 1000
# i.e time complexcity is o(n)
```

O(n^2)

```
n = 10
count = 0
for i in range(n):
    for j in range(n):
```

```

        count += 1
print(count)
#O/P:-
if n=10      count = 100
if n=100     count = 10000
if n=1000    count = 1000000
# i.e time complexcity is o(n^2)

```

O(log(n))

```

n = 100
i = 1
count = 0

while i <= n:
    count += 1
    i = i * 2

print(count)

# O/P:-
7
# hear value of i is 1 → 2 → 4 → 8 → 16 → 32 → 64 → 128(terminated)

n = 10      count = 4
n = 100     count = 7
n = 1000    count = 10

```

Explain:

i = 22222.....k-times

$i = 2^k$

loop is termination when $i \leq n$

i.e

$2^k = n$

$k = \log_2(n)$

```

i = n = 10
count = 0

while i >= 1:
    count += 1
    i = i // 2
print(count)

```

O/P:-

```
n = 10      count = 3
n = 100     count = 5
n = 1000    count = 7
```

$i = n / 3 / 3 / 3 \dots$ (k times)

$i = n / 3^k$

loop is termination when $i \leq 1$

$1 = n / 3^k$

i.e $3^k = n$

$k = \log_3(n)$

$O(n \log(n))$

```
n = 10
count = 0

for i in range(n):      # O(n)
    j = 1
    while j <= n:        # O(Log n)
        count += 1
        j = j * 2
print(count)
```

O/P:-

```
n = 10      count = 40
n = 100     count = 700
n = 1000    count = 10000
```

Outer loop runs - n times

Inner loop runs -

$i = 22222 \dots k$ -times

$i = 2^k$

loop is termination when $i \leq n$

i.e

$2^k = n$

$k = \log_2(n)$

Total = $n \times \log n$

i.e final time complexity is - **$O(n \log n)$**